

Technical Handover & Production Deployment Guide

Application: Influencer Partnership & Sales Management Platform **Repository:** Harsxch/Harsxch.github.io (main , merged from branch claude/influencer-sales-platform-q6bot0)
Audit date: 2026-09-24 (revised — the product was simplified to a two-role model since the first version of this document; see the note below) **Audited by:** direct inspection of the codebase (package files, schema, routes, auth, business logic) — not a description from memory. Anything not verifiable in code is explicitly marked **REQUIRES TECH TEAM CONFIRMATION**.

Read this if you are the engineer or team taking this application from "built by Claude Code" to "running in production." It documents what exists today, what's mock vs. real, what's missing, and exactly what you need to do/provide to go live.

What changed since the first handover: the product was deliberately simplified. The original build had 5 roles and ~20 nav pages across admin and influencer; it's now **2 roles (Admin, Influencer)** with a **3-item admin nav** (Dashboard, Influencers, Sales Tracking) and a **1-item influencer nav** (Sales Tracking). Every section below reflects the current, simplified codebase — not the original broader build.

1. Executive Summary

This is a deliberately lightweight web platform for running an influencer/creator partnership program for an online course business. It is **not** a general CRM — it does exactly two jobs: let an Admin manage influencers, and give both Admin and Influencer visibility into sales through one shared "Sales Tracking" concept (date range + UTM/coupon/course filters + summary and breakdown views).

Who uses it — exactly two roles, nothing else: - **Admin** — manages influencers (add/edit/activate/deactivate) and sees Sales Tracking across every influencer, with a breakdown by influencer, course, UTM content, and coupon. - **Influencer** — sees only their own Sales Tracking: the same date/UTM/coupon/course filters, scoped to their own sales only, enforced server-side.

The platform also still contains the fuller feature set from an earlier, broader build (commercial agreements, campaigns, tracking-link self-serve creation, coupons, payouts, earnings ledger, goals, notifications) — that code is real, tested, and still runs, but it's now **intentionally hidden from navigation** because the current product direction narrowed scope to Influencer Management + Sales Tracking only. See §2 for exactly what's nav-visible vs. hidden-but-present.

Current scope: A working, fully-implemented core product on top of **mock/manually-entered data** — there is no live payment gateway, LMS, or Metabase connection wired in yet (see §10). The financial calculation engine, RBAC, and data-isolation logic are real and production-grade in design; what's missing before go-live is almost entirely **integration** (real data sources, hosting, CI/CD, secrets) rather than application logic.

Development status: Feature-complete for the current simplified scope. No automated test suite exists beyond one manual smoke script (§23). No deployment pipeline exists yet (§15–16).

Before production launch, the tech team must: 1. Stand up a real Postgres database and hosting environment (§15–16). 2. Decide on and build the real Sales Tracking data source (currently mock data, §10). 3. Wire up whatever order-ingestion path (API/webhook/CSV) matches the real course platform — only manual entry exists today (§22 Known Limitations). 4. Rotate/generate all production secrets (§8) — do not reuse anything found in this environment's .env . 5. Add monitoring, backups, and CI (§16, §24, §25) — none exist today.

2. Product Overview

Current nav-visible scope is deliberately narrow: two things only.

Nav	Items
Admin (3 items)	Dashboard, Influencers , Sales Tracking
Influencer (1 item)	Sales Tracking

Everything else the platform can do still exists in the codebase and works (verified — see §23), it's just not linked from either nav anymore, per the product direction to keep the surface area minimal. The table below marks each module **NAV-VISIBLE** or **HIDDEN (not deleted)** accordingly.

Module	Status	Notes
Influencer management (CRUD, profile, manager assignment)	IMPLEMENTED — NAV-VISIBLE	Admin-side pages + Server Actions; one of the 2 core features
Sales Tracking — Influencer (UTM/coupon self-serve filtering)	PLACEHOLDER / MOCK — NAV-VISIBLE	Fully built UI + API contract; data is synthetically generated, not from a real DB table or Metabase (§10). The influencer's only nav item.
Sales Tracking — Admin (all-influencer view with breakdowns)	IMPLEMENTED — NAV-VISIBLE	Real DB-backed (not mock). Extended from the original admin Sales page: added an Influencer filter and by-influencer/course/UTM-content/coupon breakdowns (§11.2)
Influencer dashboard (standalone, own performance)	REMOVED FROM NAV	Folded into the Sales Tracking page's own summary cards; the old route now redirects to Sales Tracking rather than rendering separately
Tracking links (admin-issued + influencer self-serve creation)	IMPLEMENTED — HIDDEN (not deleted)	Still reachable by direct URL; Sales Tracking's UTM filters read the data this produces
Coupons	IMPLEMENTED — HIDDEN (not deleted)	Sales Tracking's coupon filter reads this data
Courses	IMPLEMENTED — HIDDEN (not deleted)	Sales Tracking's course filter reads this data
Campaigns	IMPLEMENTED — HIDDEN (not deleted)	Not referenced by Sales Tracking's current filter set beyond the underlying order data
Commercial agreements & versioning	IMPLEMENTED — HIDDEN (not deleted)	Not shown in either Sales Tracking view; still used by the earnings/payout pipeline it always was
Commission/revenue-share calculation	IMPLEMENTED	Pure decimal.js -based engine, 4 model types; runs the same as before, just not surfaced in either nav
Earnings ledger	IMPLEMENTED — HIDDEN (not deleted)	Immutable per-transaction snapshots
Payouts	IMPLEMENTED — HIDDEN (not deleted)	Bundles approved earnings; no real payment-provider integration (money movement itself is manual/external)

Module	Status	Notes
Refunds / reversals	IMPLEMENTED — HIDDEN (not deleted)	Proportional reversal against original earning, supports partial refunds
Goals	IMPLEMENTED — HIDDEN (not deleted)	Per-influencer target tracking
Notifications	IMPLEMENTED — HIDDEN (not deleted)	In-app only
Marketing assets	PARTIALLY IMPLEMENTED — HIDDEN (not deleted)	Data model + admin listing exist; no file upload — asset URLs are pasted in, not uploaded
Audit log	IMPLEMENTED — HIDDEN (not deleted)	Append-only by construction, no update/delete path in the app
CSV exports (sales/earnings/payouts/influencers)	IMPLEMENTED	Synchronous, capped at 10,000 rows — flagged as a scale limit in existing docs; the sales export button is still on the admin Sales Tracking page
Order ingestion via real payment/LMS webhook or API	NOT IMPLEMENTED	Only manual entry by admin staff exists; schema has <code>OrderSource.API / WEBHOOK / CSV_IMPORT</code> values reserved for later
Email/WhatsApp notification delivery	NOT IMPLEMENTED	In-app notifications only
Payment gateway integration	NOT IMPLEMENTED	No Stripe/Razorpay/etc. anywhere in the code
Metabase integration	NOT IMPLEMENTED	See §10
File upload (images/documents)	NOT IMPLEMENTED	No upload endpoint exists
Automated tests / CI	NOT IMPLEMENTED	One manual Playwright smoke script only, re-verified passing after the simplification (§23)

3. User Roles & Access

Roles (`Role` enum): `ADMIN` , `INFLUENCER` — **exactly two, by design**. This is a real database-level change, not a UI relabeling: a migration (`20260924000000_simplify_roles_to_admin_influencer` , §9) collapsed the original 5-role model (`SUPER_ADMIN` , `FINANCE` , `INFLUENCER_MANAGER` , `ANALYST` , `INFLUENCER`) down to these 2, casting every former admin-tier role to `ADMIN` and leaving `INFLUENCER` unchanged. The old distinctions between finance/management/analyst back-office functions no longer exist anywhere in the app — one Admin account does everything on the back-office side.

Role	Can access	Can create/edit	Cannot access
ADMIN	Everything: influencers, courses, campaigns, coupons, agreements, orders, refunds, transactions, payouts, assets, goals, audit logs, staff <code>users</code> , exports	Everything listed	—
INFLUENCER	Only their own data: own profile, own assigned courses, own tracking links (read+write, self-serve), own coupons, own orders (PII-stripped), own ledger, own payouts, own goals, own Sales Tracking	Own tracking links only	Any other influencer's data, all admin resources, staff users, audit logs

Influencer data isolation — how it's enforced (this is the critical security property of the app, and it did not change or weaken in the role simplification):

There are **two independent, server-side layers**, both must pass on every request — the frontend never enforces this alone:

- Resource-level RBAC** — `src/lib/auth/rbac.ts` . `assertCan(role, resource, action)` throws a `ForbiddenError` (→ HTTP 403) if the role lacks permission for that resource/action at all.
- Row-level scoping** — `src/lib/auth/session.ts` : `resolveInfluencerScope(sessionUser, requestedInfluencerId)` — if the caller's role is `INFLUENCER` , this **always** returns the session's own `influencerId` and silently ignores whatever id was requested. `assertOwnInfluencerScope(sessionUser, requestedInfluencerId)` — throws `ForbiddenError("Cannot access another influencer's data")` if an `INFLUENCER` caller's session id doesn't match the requested id.

The **Sales Tracking** feature (the newest, and the one this handover's manager-facing sibling document focuses on) goes one step further: `src/lib/sales-tracking/service.ts` derives the influencer's identity **exclusively** from the server session and does not accept an influencer/user id as an input parameter **at all** — there is no field for a client to tamper with in the first place. This was a specific, explicit security requirement during development and was verified: a logged-in influencer hitting `/api/influencer/sales-tracking` cannot pass any parameter to see another influencer's data; a non-INFLUENCER role gets a 403.

Status: this enforcement is server-side, not just frontend-hidden. No security issue was found in this area during the audit. The one general caveat: `assertCan / assertOwnInfluencerScope` usage was verified directly in the sales-tracking, dashboard, and ledger data-access functions; it was **not independently re-verified line-by-line in every single `actions.ts` file** across the whole admin/influencer tree in this pass — recommend the tech team grep for any `src/lib/data/*.ts` or `actions.ts` function that queries by an id **without** first calling `resolveInfluencerScope / assertOwnInfluencerScope` as part of their own pre-launch security pass (§16).

4. Application Architecture

Diagram (Mermaid source — renders visually on GitHub/most Markdown viewers):

```
graph TD
    U[User: Admin / Influencer] --> FE[Next.js App Router - Server + Client Components]
    FE --> MW[Edge Middleware - src/middleware.ts]
    MW --> AUTH[Auth.js v5 - JWT session]
    AUTH --> RSC[Server Components / Server Actions]
    RSC --> RBAC[RBAC layer - rbac.ts]
    RBAC --> SCOPE[Row-level scoping - session.ts]
    SCOPE --> LIB[Business logic - src/lib]
    LIB --> PRISMA[Prisma 7 Client + adapter-pg]
    PRISMA --> PG[(PostgreSQL 16)]

    FE -.5 read-only API routes. .-> API[src/app/api/**]
```

```
API --> RBAC
```

```
subgraph Sales Tracking specifically
  ST[Influencer Sales Tracking page] --> STAPI[/api/influencer/sales-tracking/]
  STAPI --> STSVC[sales-tracking/service.ts]
identity from session ONLY]
  STSVC --> STADAPTER[SalesTrackingAdapter interface]
  STADAPTER --> MOCK[MockSalesTrackingAdapter]
CURRENT: synthetic in-memory data]
  STADAPTER -.NOT YET BUILT.-> METABASE[(Metabase / real data source)]
end
```

- **Frontend & Backend:** same Next.js 16 App Router app — Server Components render pages, Server Actions handle almost all writes, a small set of route handlers (`src/app/api/**`) handle reads that need to be called from client-side `fetch` or return CSV.
- **Database:** single PostgreSQL 16 instance, accessed only through Prisma.
- **External services:** none currently (§9, §10).
- **Storage:** none — no file upload capability exists.
- **Background processes:** none — no cron/queue/worker of any kind.

5. Technology Stack

Technology	Version	Purpose	Notes
Next.js	16.3.5	Full-stack framework (App Router, Server Components/Actions, Turbopack)	
React / React DOM	19.2.8	UI runtime	
TypeScript	5.9.3	Language	
PostgreSQL	16	Primary database	Confirmed via local dev cluster (<code>pg_lsclusters</code>); production version REQUIRES TECH TEAM CONFIRMATION
Prisma	7.10.0 (<code>prisma + @prisma/client</code>)	ORM / schema / migrations	Uses the newer <code>prisma-client</code> generator with a custom output path <code>src/generated/prisma</code> (not the legacy <code>prisma-client-js</code>)
@prisma/adapters/pg	^7.10.0	Driver adapter required by Prisma 7's new client engine	Wraps <code>pg</code>
pg	^8.23.0	Postgres driver	
next-auth (Auth.js)	5.0.0-beta.32	Authentication, JWT sessions	Beta version — see §14
bcryptjs	^3.0.3	Password hashing	
Tailwind CSS	^4 (resolved 4.3.3)	Styling	
decimal.js	^10.6.0	Arbitrary-precision decimal math for all money calculations	Never floating point
zod	^4.6.5	Runtime input validation (Server Action / order schemas)	
lucide-react	^1.47.0	Icon set	
recharts	^3.10.1	Charts (dashboard revenue/earnings graphs)	
Playwright	^1.63.0 (dev dependency)	Browser automation	Used only for one ad hoc smoke script, not a configured test runner (§23)
tsx	^4.23.15 (dev)	Run TypeScript scripts directly (seed script)	
ESLint	^9 + eslint-config-next 16.3.5 (dev)	Linting	
npm	—	Package manager	<code>package-lock.json</code> present; no <code>pnpm/yarn</code> lockfile

No file storage/CDN library, no payment SDK, no email/SMS SDK, no error-tracking SDK (e.g. Sentry), and no logging library are present in the dependency tree.

6. Project Structure

Everything lives under `src/` (no top-level `/app`). Path alias `@/*` → `./src/*` .

```
/
├── .env.example           # committed, placeholders only
├── README.md             # quickstart + stack summary
├── docs/
│   ├── ARCHITECTURE.md   # design rationale, referenced by section number to an external spec
│   ├── TECHNICAL_HANOVER.md # this document
│   └── screenshots/      # screenshots referenced by this document
├── prisma.config.ts      # Prisma 7 CLI config (schema/migrations paths, DATABASE_URL)
├── prisma/
│   ├── schema.prisma     # full data model (§9) – Role enum is now just ADMIN | INFLUENCER
│   ├── seed.ts           # dev/demo data generator – npm run db:seed
│   └── migrations/       # 3 migrations to date, the 3rd collapses the role model
├── scripts/
│   └── e2e-smoke.mjs      # manual Playwright smoke script (§23)
├── legacy-portfolio/     # UNRELATED – leftover static portfolio site from this repo's origin as a personal GitHub Pages site. Confirm with the team whether it can be deleted.
```

```
└─ src/
  └─ middleware.ts      # coarse role-based route gate (role !== "ADMIN" / !== "INFLUENCER")
  └─ generated/prisma/  # Prisma Client output (generated; not hand-edited)
  └─ app/
    └─ login/          # login page + Server Action
    └─ api/             # the ONLY 5 route handlers in the app (§12)
      └─ auth/[...nextauth]/route.ts
      └─ exports/{earnings,influencers,payouts,sales}/route.ts
      └─ influencer/sales-tracking/route.ts
    └─ r/[code]/route.ts # public tracking-link redirect (not under /api)
    └─ admin/           # staff-facing pages + colocated Server Actions - nav.ts only links Dashboard, Influencers, Sales Tracking; the rest
                        (Courses, Campaigns, Tracking Links, Coupons, Agreements, Earnings, Payouts, Audit Logs) still exist as routes, just unlinked
  └─ influencer/        # influencer-facing pages + colocated Server Actions - nav.ts links only Sales Tracking; the old dashboard route now just
                        redirects there
  └─ components/
    └─ charts/          # Recharts wrappers
    └─ filters/         # shared sales filter bar (now used by both admin and influencer Sales Tracking, with an added Influencer dropdown on
                        the admin side)
  └─ layout/           # app-shell (sidebar/nav), sign-out button
  └─ ui/               # badge, button, card, table, stat-card, etc.
  └─ lib/              # ALL business logic — the only place that talks to Prisma
    └─ auth/           # auth.config.ts, auth.ts, auth-edge.ts, rbac.ts, session.ts - rbac.ts's matrix now has exactly 2 roles
    └─ financial/       # calculation engine, agreements, payouts, reversal
    └─ attribution/     # coupon/tracking-link attribution strategies
    └─ orders/          # order + refund ingestion pipeline (manual-entry only); also where the admin Sales Tracking aggregate query lives
    └─ sales-tracking/  # the influencer Sales Tracking feature (§10–11)
    └─ data/            # per-domain read queries (each enforces RBAC + scoping)
    └─ coupons/, audit/, notifications/, api/, csv.ts, format.ts, date-range.ts, prisma.ts
```

A new engineer mainly needs: `src/lib/auth/` (how identity/permissions work — now a 2-role matrix), `src/lib/financial/` (how money is calculated), `src/lib/sales-tracking/` (the influencer feature needing real data), `src/lib/data/orders.ts` (the admin Sales Tracking aggregate), `prisma/schema.prisma` (the data model), and the relevant `src/app/**` folder for whatever page they're changing.

7. Local Development Setup

Exact steps, taken from `package.json` and `README.md` (no invented commands):

```
# 1. Prerequisites
# - Node.js (v22.x confirmed working in this environment; exact minimum REQUIRES TECH TEAM CONFIRMATION — no `engines` field is set)
# - PostgreSQL 16 reachable locally
# - npm

# 2. Install dependencies
npm install

# 3. Configure environment
cp .env.example .env
# then fill in real values (see §8)

# 4. Apply database schema
npx prisma migrate deploy

# 5. (Optional) seed demo/dev data
npm run db:seed

# 6. Run the dev server
npm run dev # http://localhost:3000

# 7. Build for production
npm run build
npm start

# 8. Lint
npm run lint

# 9. Manual smoke test (requires steps 4–6 already done)
npm run e2e:smoke
```

There is **no** `npm test` script and **no** automated unit/integration test command (§23).

Seeded demo logins (from `prisma/seed.ts`, password `Passw0rd!` for all) — only 2 roles now:

Email	Role
admin@platform.dev	ADMIN
rahul@creator.dev / priya@creator.dev / amina@creator.dev	INFLUENCER

8. Environment Variables

Variable	Purpose	Required?	Example/Format	Used in	Production Notes
<code>DATABASE_URL</code>	Postgres connection	Yes	<code>postgresql://user:YOUR_SECRET_HERE@host:5432/dbname?schema=public</code>	<code>src/lib/prisma.ts</code> , <code>prisma/seed.ts</code>	Must point at the production

Variable	Purpose	Required?	Example/Format	Used in	Production Notes
	string for Prisma			<code>prisma.config.ts</code>	Postgres instance; use a pooled connection string if the host requires one (e.g. pgbouncer) — REQUIRES TECH TEAM CONFIRMATION on the chosen host's pooling requirements
<code>AUTH_SECRET</code>	JWT signing secret for Auth.js sessions	Yes	<code>YOUR_SECRET_HERE</code> (generate via <code>openssl rand -base64 32</code>)	Read implicitly by NextAuth v5 internals	Must be a fresh, random value in production — never reuse a dev value
<code>NEXTAUTH_URL</code>	Canonical app URL, used by Auth.js for callback construction	Yes	<code>https://your-production-domain.com</code>	Read implicitly by NextAuth v5 internals	Must exactly match the production domain (including protocol)
<code>NODE_ENV</code>	Standard Node environment flag	Set automatically by <code>next build / next start</code>	<code>production</code>	<code>src/lib/prisma.ts</code> (guards the dev-only hot-reload PrismaClient caching pattern)	No action needed — Next.js sets this
<code>BASE_URL</code>	Target URL for the manual smoke script only	No (defaults to <code>http://localhost:3000</code>)	<code>https://staging.your-domain.com</code>	<code>scripts/e2e-smoke.mjs</code>	Not needed at runtime, only if the tech team runs the smoke script against a deployed environment
<code>PLAYWRIGHT_EXECUTABLE_PATH</code>	Optional Chromium path override for the smoke script	No	path to a Chromium binary	<code>scripts/e2e-smoke.mjs</code>	Only relevant in sandboxed CI environments where Playwright can't download its own browser

No frontend-exposed (`NEXT_PUBLIC_*`) variables exist. No Metabase, payment, email/SMS, or storage-related environment variables exist because those integrations are not implemented (\$2, \$10).

Security note found during this audit: a real `.env` file with live-looking `DATABASE_URL` / `AUTH_SECRET` / `NEXTAUTH_URL` values exists on disk in the development environment this app was built in. It is correctly excluded via `.gitignore` and was **not** committed to the repository (verified with `git ls-files / git check-ignore`). Treat any values in that file as compromised for a shared/dev database — generate fresh secrets for every new environment (dev, staging, production) rather than copying it forward.

9. Database / Data Architecture

Type: PostgreSQL 16, accessed only through Prisma 7 (`@prisma/adpater-pg` driver adapter). Schema at `prisma/schema.prisma` (3 migrations applied to date: `20260922134735_init` , `20260922155721_tracking_link_utm_term_and_coupon` , `20260924000000_simplify_roles_to_admin_influencer` — the third collapses the 5-role model down to `ADMIN` / `INFLUENCER` with a safe data cast, verified to replay cleanly against a fresh database).

All money fields are `Decimal(12,2)` (Postgres `NUMERIC`) — never floating point — matching the `decimal.js`-based calculation engine.

Core relationship chain

Diagram (Mermaid source — renders visually on GitHub/most Markdown viewers):

```
graph LR
    User -->|1:1| Influencer
    Influencer -->|has many, versioned| CommercialAgreement
    CommercialAgreement -->|has many, immutable| CommercialAgreementVersion
    Influencer --> TrackingLink
    Influencer --> Coupon
    Campaign --> TrackingLink
    Campaign --> Coupon
    Course --> TrackingLink
    Course --> Coupon
    TrackingLink -- attribution --> OrderAttribution
    Coupon -- attribution --> OrderAttribution
    Order -->|1:1| OrderAttribution
    Order --> FinancialTransaction
    CommercialAgreementVersion -- snapshot --> FinancialTransaction
    FinancialTransaction --> Payout
    Order --> Refund
    Refund -- creates REVERSAL --> FinancialTransaction
```

Key entities (selected — full model list is in `prisma/schema.prisma`)

Entity	Key fields	Notable constraints
User	email, passwordHash, role, isActive	email unique; @@index([role])
Influencer	userId (1:1), managerId → User, status	email unique; indexed on status, managerId
CommercialAgreement / CommercialAgreementVersion	modelType, shares/commission/fixedAmount, effectiveFrom/Until, status	Versions are immutable and append-only; @@unique([agreementId, version])
TrackingLink	code (unique), influencerId, courseId, utmSource/Medium/Campaign/Content/Term, couponId	Indexed on influencerId, courseId, status
Coupon	code (unique), influencerId, discountType/Value, agreementVersionId (override), usageLimit/currentUsage	Indexed on influencerId, status
Customer	email (unique), name, phone	Deliberately minimal — "influencers must never see customer PII" (schema comment, line ~497)
Order	orderId (unique), customerId, courseId, couponId, originalPrice/discountAmount/finalAmount	Indexed on courseId, status, placedAt, couponId
OrderAttribution	orderId (1:1, unique), influencerId, trackingLinkId, couponId, source	Indexed on influencerId, trackingLinkId, couponId
FinancialTransaction	type, status, immutable snapshot fields of the agreement terms used, eligibleRevenue/influencerAmount/companyAmount	Indexed on [influencerId, status] , orderId, payoutId; self-relation for REVERSAL → EARNING linkage
Payout	influencerId, amount, status, reference, method	Indexed on [influencerId, status]
AuditLog	action, entityType/Id, previousValue/newValue (JSON)	Append-only by application-layer convention — no update/delete path exists in the code

Full enum list: Role , InfluencerStatus , AgreementModelType , EligibleRevenueBasis , AgreementStatus , CourseStatus , CampaignStatus , TrackingLinkStatus , CouponDiscountType , CouponStatus , OrderStatus , OrderSource , AttributionSource , TransactionType , TransactionStatus , PayoutStatus , GoalMetric , AssetType , NotificationType .

10. Metabase / Data Source Integration

Metabase integration is not currently implemented in the application. A repo-wide case-insensitive search for "metabase" returns matches only inside code comments in `src/lib/sales-tracking/` describing it as a **future** integration point — there is no Metabase SDK dependency, no API base URL, no connection string, and no HTTP call to Metabase anywhere in the code.

CURRENT IMPLEMENTATION (Sales Tracking data)

The Sales Tracking feature is built against a clean adapter interface specifically so a real data source can be dropped in later without touching the UI or API route:

```
// src/lib/sales-tracking/adapter.ts
export interface SalesTrackingAdapter {
  getFilterOptions(identity: InfluencerIdentity): Promise<SalesFilterOptions>;
  getSales(identity: InfluencerIdentity, filters: SalesFilters): Promise<InfluencerSalesResponse>;
}

// src/lib/sales-tracking/index.ts <-- THE SINGLE SWAP POINT
export const salesTrackingAdapter: SalesTrackingAdapter = new MockSalesTrackingAdapter();
```

Today, `MockSalesTrackingAdapter` (`src/lib/sales-tracking/mock-adapter.ts`) generates **deterministic synthetic data in memory** (`mock-data.ts` — seeded per-influencer-id PRNG, 35–54 fake sales rows from fixed course/UTM/coupon vocab lists, cached per server process). There is no database table backing it and no real order data is read.

EXPECTED PRODUCTION DATA INTEGRATION

To go live, the tech team implements a new class satisfying the same `SalesTrackingAdapter` interface (e.g. `MetabaseSalesTrackingAdapter`) and changes the one export in `index.ts` . Nothing else in the app needs to change.

API endpoint the frontend calls: `GET /api/influencer/sales-tracking`

Query parameters (all optional; absence = no filter applied):

Param	Type	Example
startDate	ISO date string	2026-06-01
endDate	ISO date string	2026-09-30
utmSource	string	youtube
utmMedium	string	influencer
utmCampaign	string	dedicated
utmContent	string	sf
utmTerm	string	HarshPriyam
couponCode	string	HPDSPAPP
courseName	string	Python + DSA Mastery

Note: identity (which influencer) is **never** a parameter — it comes only from the server-side session (§3).

Response shape (`InfluencerSalesResponse`):

```
{
  "summary": {
    "totalSales": 51,
    "totalRevenue": 597800,
    "coursesSold": 3
  },
  "courseBreakdown": [
    { "courseName": "AI Mastery", "sales": 16, "revenue": 232500 }
  ],
  "utmBreakdown": [
    { "utmContent": "sf", "sales": 13, "revenue": 155050 }
  ],
  "sales": [
    {
      "orderId": "ORD-MUDJS9YP-WC28",
      "date": "2026-09-18",
      "courseName": "Python + DSA Mastery",
      "utmSource": "storefront",
      "utmMedium": "V_Upskill_Academy",
      "utmCampaign": "Dedicated",
      "utmContent": "sf",
      "utmTerm": "HarshPriyam",
      "couponCode": "HPDSPAPP",
      "userId": "usr_9f2ab1",
      "saleAmount": 10000,
      "saleStatus": "PAID"
    }
  ]
}
```

Required fields per sale row: `orderId`, `date`, `courseName`, `userId`, `saleAmount`, `saleStatus`. Optional/nullable: `utmSource`, `utmMedium`, `utmCampaign`, `utmContent`, `utmTerm`, `couponCode`. `saleStatus` is one of `"PAID"` | `"PENDING"` | `"REFUNDED"` | `"CANCELLED"`.

11. Sales Tracking Feature (Influencer + Admin)

Sales Tracking is now the platform's core function for both roles — this is intentional, per the simplified product direction. It's two separate pages built on shared underlying data (`Order` / `OrderAttribution`), not one shared page: the influencer sees only their own sales, the admin sees everyone's with an added influencer breakdown.

11.1 Influencer Sales Tracking

Access: Influencer logs in → lands directly on "Sales Tracking," their only nav item.

Flow:

Diagram (Mermaid source — renders visually on GitHub/most Markdown viewers):

```
flowchart LR
  A[Logged-in Influencer] --> B[Sales Tracking page loads  
empty state - no fetch yet]
  B --> C[Pick date range + optional UTM/coupon/course filters]
  C --> D[Click Apply Filters]
  D --> E[GET /api/influencer/sales-tracking]
  E --> F[service.ts resolves identity from session ONLY]
  F --> G[salesTrackingAdapter.getSales]
  G --> H[Mock adapter today / Metabase adapter later]
  H --> I[Summary cards + course breakdown + UTM breakdown + detail table]
```

- **Filters:** Start Date & End Date (both required before any fetch happens), UTM Source, UTM Medium, UTM Campaign, UTM Content, UTM Term, Coupon Code, Course Name (all optional, populated dynamically per-influencer from `getFilterOptions`).
- **States:** idle (default — "select a date range," no data loaded), loading (spinner), error ("unable to load, retry" button), loaded-empty ("no sales found for the selected filters"), loaded-with-data.
- **Data shown on success:** 3 summary cards (Total Sales, Total Revenue, Courses Sold), a course-wise breakdown table, a UTM-content breakdown table, and a full sales detail table (date, course, all UTM fields, coupon, opaque User ID, order ID, amount, status).
- **Reset:** "Clear Filters" resets all filters and returns to the idle state.
- **Pagination: NOT IMPLEMENTED** — the detail table renders all matching rows at once. **Export: NOT IMPLEMENTED** for this specific feature (the separate `/api/exports/sales` admin-only CSV export is unrelated and not influencer-facing).
- Runs on mock data — see §10 for the real-data integration path. Unlike the admin view below, this one is not backed by the real database at all yet.

11.2 Admin Sales Tracking

Access: Admin logs in → "Sales Tracking" in the 3-item nav (`/admin/sales`, extended from the original admin Sales page rather than built new).

Unlike the influencer view, this one is backed by real database data (the `Order` / `OrderAttribution` / `FinancialTransaction` tables) — it's the same order data the manual-entry pipeline (§13, §20) writes to.

- **Filters:** Start Date, End Date, Course, Campaign, **Influencer** (new — a dropdown of all influencers), Coupon Code, Status, UTM Source, UTM Medium, UTM Campaign, UTM Content, UTM Term. Same shared `SalesFilterBar` component the influencer page uses, with the Influencer dropdown added only on this side.
- **Data shown:** Total Sales / Total Revenue summary cards, four breakdown tables — **by influencer**, by course, by UTM content, by coupon (each: count + revenue, sorted by revenue descending) — and the existing paginated order detail table below (unchanged from the original build: order ID, date, course, influencer, coupon, UTM fields, amount, earnings, status).
- **Implementation:** `getSalesTrackingSummary()` in `src/lib/data/orders.ts`, sharing its `where` -clause builder with the existing `listOrders()` so the breakdowns and the detail table are always filtering the exact same set of orders. Aggregation is done in application code over up to 10,000 matching orders (same cap as the CSV exports) — not a SQL `GROUP BY` — acceptable at current data volumes but worth revisiting if order volume grows large (§22).
- **Pagination:** the detail table is paginated (25/page) as before; the summary cards and breakdown tables are not (they aggregate across every matching order, not just the current page).
- **Export:** the existing "Export CSV" button (`/api/exports/sales`) still works and respects the same filters, including the new influencer filter.

Screenshots for both views are in §19.

12. API Documentation

All 5 route handlers in the app, verbatim from `src/app/api/**/route.ts`. Everything else (agreements, campaigns, coupons, courses, orders, payouts, tracking-links CRUD) is implemented as **Server Actions**, not REST endpoints — there is no separate JSON API for those.

GET/POST `/api/auth/[...nextauth]`

NextAuth's own internal machinery (signin/callback/session/csrf). Not hand-written; no custom logic to document.

GET `/api/influencer/sales-tracking`

- **Auth:** session required; role must be `INFLUENCER` with a resolved `influencerId`, or 403.
- **Authorization:** identity comes only from the session — no id parameter accepted.
- **Query params / response:** see §10 (Metabase / Data Source Integration) above.
- **Errors:** 401 not authenticated, 403 wrong role, 500 unexpected error (all via the shared `apiHandler` wrapper).

GET `/api/exports/earnings`

- **Auth:** session + `assertCan(role, "exports", "read")` — `ADMIN` only (the only role that can hold this permission now).
- **Query params:** `status` (transaction status filter — passed through without runtime validation against the enum).
- **Response:** CSV file (`earnings.csv`).
- **Note:** no row-level influencer scoping is applied — any permitted role receives ALL influencers' earnings.

GET `/api/exports/influencers`

- **Auth:** same `assertCan` pattern.
- **Query params:** none.
- **Response:** CSV (`influencers.csv`), capped at 10,000 rows.

GET `/api/exports/payouts`

- **Auth:** same pattern. **Query params:** none. **Response:** CSV (`payouts.csv`).

GET `/api/exports/sales`

- **Auth:** same pattern.
- **Query params:** `status`, `courseId`, `campaignId`, `influencerId`, `couponCode`, `utmSource`, `utmMedium`, `utmCampaign`, `utmContent`, `utmTerm`, `from`, `to` — all forwarded to `listOrders()`.
- **Response:** CSV (`sales.csv`).

GET `/r/[code]` (public, not under `/api`)

- **Auth:** none — intentionally public.
- **Purpose:** tracking-link redirect. Looks up the `TrackingLink` by `code`; if missing/inactive, redirects to `/`; otherwise increments the click counter and 302-redirects to the course URL with the link's full UTM parameters and coupon code appended.

13. Authentication

- **Login flow:** single email+password Credentials provider (Auth.js v5 / NextAuth). `authorize()` (`src/lib/auth/auth.ts`) looks up the user by lowercased/trimmed email, rejects inactive users, compares the password with `bcrypt.compare`, updates `lastLoginAt`, and returns `{ id, email, name, role, influencerId }`.
- **Session management:** JWT strategy (no database session table). Role and `influencerId` are copied onto the JWT at sign-in and exposed on `session.user` via the session callback.
- **Token management:** handled entirely by Auth.js's own JWT encode/decode using `AUTH_SECRET`; no custom token logic.
- **Password handling:** bcryptjs hash, never stored/logged in plaintext.
- **Identifying the logged-in influencer:** every server-side function calls `requireSession()` (`src/lib/auth/session.ts`) which is the sole source of truth for “who is calling”; `session.user.influencerId` is what ties a request to a specific `Influencer` row — never a client-supplied parameter.
- **Logout:** standard Auth.js sign-out (`SignOutButton` component).
- **Session expiration:** governed by Auth.js's default JWT session settings — no custom `maxAge` /rotation was found configured; **exact expiry duration REQUIRES TECH TEAM CONFIRMATION** if a specific policy is needed.
- **Password reset: NOT IMPLEMENTED** — no reset-flow pages, tokens, or email sending exist.
- **Edge/Node split:** Next.js Edge Middleware can't load the generated Prisma client, so auth is split into `auth.config.ts` (edge-safe, stub `authorize`), `auth.ts` (full config with the real `Prisma/bcrypt authorize`, used everywhere except middleware), and `auth-edge.ts` (edge-safe `auth()` used only by `middleware.ts`).

14. Security Review

Area	Finding
Exposed secrets / hardcoded credentials	None found in source. A real <code>.env</code> exists on disk in this dev environment (correctly git-ignored) — rotate before any shared use (§8, §10).
Unsafe API endpoints	All 5 route handlers are wrapped in <code>apiHandler</code> with explicit auth checks; none found unauthenticated except the intentionally-public <code>/r/[code]</code> redirect.
Missing authorization	Not found in the paths independently verified (sales-tracking, dashboard, ledger). Not exhaustively re-verified across every Server Action — recommend a full grep pass before launch (§3).
Client-side-only access control	None found — the influencer data-isolation property is enforced server-side in two independent layers (§3), not just hidden in the UI.
SQL injection	Low risk — all queries go through Prisma's parameterized query builder; no raw SQL string concatenation was found.
XSS	Low risk — standard React/JSX escaping throughout; no <code>dangerouslySetInnerHTML</code> usage found.
CSRF	Handled by Auth.js's built-in CSRF protection for its own endpoints; Server Actions get Next.js's built-in Origin-header protection. Not independently penetration-tested.

Area	Finding
Insecure direct object references (IDOR)	Sales Tracking specifically has no id parameter to tamper with at all (identity is session-derived). Admin export routes (<code>/api/exports/sales</code> etc.) accept an <code>influencerId</code> filter, but access to those routes is already gated to admin-tier roles by RBAC — not a per-influencer IDOR risk since influencers can't reach them.
Excessive API responses	Admin-only CSV exports return full unfiltered rows (capped at 10,000) to any role with <code>exports</code> permission — by design, those roles are meant to see all data.
Unsafe file uploads	N/A — no upload functionality exists.
Debug endpoints / dev credentials	<code>prisma/seed.ts</code> creates known dev credentials (<code>Passw0rd!</code> for all seeded accounts) — must not be run against production , or if it is for initial setup, all seeded passwords must be rotated immediately.
CORS	No custom CORS configuration found in <code>next.config.ts</code> (default/empty config) — Next.js's same-origin defaults apply.
Production configuration issues	No environment-specific config split was found beyond <code>NODE_ENV</code> ; <code>next-auth</code> is on a beta version (<code>5.0.0-beta.32</code>) — confirm this is an acceptable production dependency before launch.

CRITICAL BEFORE PRODUCTION

- [] Rotate every secret found in this development environment's `.env` — do not carry any of them into staging/production.
- [] Run a full grep for every `src/lib/data/*.ts` function and `admin/influencer actions.ts` file to confirm `assertOwnInfluencerScope / resolveInfluencerScope` is called before any influencer-scoped query (only a sample was independently re-verified in this audit).
- [] Decide whether to pin `next-auth` off its current beta version before production, or explicitly accept the beta as final for launch.
- [] Ensure `npm run db:seed` (which creates known dev passwords) is never run against the production database, or that all seeded accounts have their passwords rotated immediately if it is used for initial data setup.

15. Deployment Architecture

CURRENT CONFIGURATION

- `next.config.ts` is empty/default — no custom rewrites, headers, image domains, or output mode set.
- No `vercel.json`, no `Dockerfile`, no `docker-compose.yml`, no `.github/workflows/` — **no deployment or CI configuration of any kind exists in the repo.**
- README explicitly states the app **cannot** run on GitHub Pages (needs a database, server-side auth, and the financial calculation engine — none of which a static host can provide) and suggests a Node-capable host (Vercel/Railway/Render/Fly.io, unconfirmed which one this org will use) with a managed Postgres provider (Neon/Supabase/Railway/RDS, likewise unconfirmed).
- Documented (README) manual deploy steps: `npm install` → configure `.env` → `npx prisma migrate deploy` → (`npm run db:seed` optionally, dev only) → `npm run build` → `npm start`.

RECOMMENDED PRODUCTION CONFIGURATION — REQUIRES TECH TEAM CONFIRMATION

The following all depend on decisions only the tech team can make; nothing below is inferred from the code: - **Frontend/Backend hosting**: any Node.js-capable host (this is a single full-stack Next.js app, not a static site) — **REQUIRES TECH TEAM CONFIRMATION.** - **Database hosting**: managed Postgres 16-compatible service — **REQUIRES TECH TEAM CONFIRMATION.** - **Domain / SSL**: not configured anywhere in the repo — **REQUIRES TECH TEAM CONFIRMATION.** - **Migrations on deploy**: `run npx prisma migrate deploy` as a release step before `npm start` ; no automation for this exists yet. - **Auth callback URLs**: `NEXTAUTH_URL` must be set to the real production domain. - **CORS / webhooks / storage / CDN**: not applicable today since none of those integrations exist yet (\$2, \$9, \$10).

16. Production Deployment Checklist

```
[ ] Production repository/branch strategy confirmed
[ ] Production hosting environment chosen and created (frontend + Node runtime)
[ ] Production PostgreSQL 16+ database provisioned
[ ] Environment variables configured (DATABASE_URL, AUTH_SECRET, NEXTAUTH_URL) — see $8
[ ] All secrets rotated from dev values (never reused from this environment)
[ ] Database migrations executed (`npx prisma migrate deploy`)
[ ] Prisma Client generated as part of the build/install step
[ ] Real Sales Tracking data adapter implemented and swapped in ($10) — or explicitly deferred with stakeholders' sign-off that mock data ships at launch
[ ] Authentication domain/callback URL configured for production
[ ] Domain configured and DNS pointed
[ ] SSL/TLS active
[ ] Application deployed and reachable
[ ] Seeded demo/dev accounts removed or passwords rotated if seed script was used for initial setup
[ ] Admin access tested end-to-end (login, Dashboard, Influencers, Sales Tracking)
[ ] Influencer access tested end-to-end (login lands on Sales Tracking directly)
[ ] Influencer data-isolation re-tested against production data (cannot access another influencer's data via any parameter)
[ ] Sales Tracking tested against whatever data source is live at launch
[ ] UTM filtering, coupon filtering, course filtering all tested individually and combined
[ ] Empty-result and error states tested
[ ] Mobile responsiveness spot-checked (already verified at 1440px/390px during development; tablet width was explicitly descope'd)
[ ] A full security pass completed (see $14 Critical list)
[ ] Logging/error visibility configured beyond default stdout capture
[ ] Monitoring configured ($26 — nothing exists today)
[ ] Backup strategy confirmed for the production database ($27 — nothing exists today)
[ ] Rollback strategy confirmed for both app deploys and DB migrations ($27)
[ ] CI pipeline set up (none exists today) running at minimum `npm run lint` and `npm run build`
```

17. Information Required From Tech Team

Only items the application genuinely needs before it can go live — not a generic wishlist:

- Production `DATABASE_URL` (host, credentials, pooling requirements).
- Production `AUTH_SECRET` and confirmed production domain for `NEXTAUTH_URL` .

- Decision + implementation plan for the real Sales Tracking data source (Metabase connection details, or whatever the actual system of record is) — see §10 for the exact interface it must satisfy.
- Real order/sale data schema mapping so a real `SalesTrackingAdapter` (or the underlying order-ingestion pipeline) can be written — see §18 Data Mapping.
- Decision on which order-ingestion path (real payment/LMS API, webhook, or scheduled CSV import) will replace today's manual-entry-only flow, and access/credentials for that source.
- Hosting environment and deployment process the org wants to standardize on (nothing is currently assumed).
- Any organizational requirement for password reset, SSO, or MFA (none of these exist today — confirm whether they're needed for launch).

18. Data Mapping (Sales Tracking)

Frontend field	API/type field	Source field (real system)	Data type	Example
Start Date / End Date	<code>startDate / endDate</code>	REQUIRES TECH TEAM CONFIRMATION (likely order placement date)	ISO date string	2026-06-01
UTM Source	<code>utmSource</code>	REQUIRES TECH TEAM CONFIRMATION (likely tracking-link/campaign UTM field in the real ad/analytics system)	string null	storefront
UTM Medium	<code>utmMedium</code>	REQUIRES TECH TEAM CONFIRMATION	string null	V_Upskill_Academy
UTM Campaign	<code>utmCampaign</code>	REQUIRES TECH TEAM CONFIRMATION	string null	Dedicated
UTM Content	<code>utmContent</code>	REQUIRES TECH TEAM CONFIRMATION	string null	sf
UTM Term	<code>utmTerm</code>	REQUIRES TECH TEAM CONFIRMATION	string null	HarshPriyam
Coupon Code	<code>couponCode</code>	REQUIRES TECH TEAM CONFIRMATION (likely the coupon/discount-code field on the order record)	string null	HPDSPAPP
Course Name	<code>courseName</code>	REQUIRES TECH TEAM CONFIRMATION (course/product name or ID in the real catalog)	string	Python + DSA Mastery
User ID	<code>userId</code>	REQUIRES TECH TEAM CONFIRMATION — must be an existing opaque, non-PII internal identifier; never phone/email/name	string	usr_9f2ab1
Sale Amount	<code>saleAmount</code>	REQUIRES TECH TEAM CONFIRMATION (final/net amount charged, matching whatever revenue basis the business reports on)	number	10000
Sale Status	<code>saleStatus</code>	REQUIRES TECH TEAM CONFIRMATION — must map onto <code>"PAID" \ "PENDING" \ "REFUNDED" \ "CANCELLED"</code>	enum string	PAID

This app's internal `Order / OrderAttribution` schema (§9) already models UTM source/medium/campaign/content/term and coupon linkage for the **manual-entry** flow that exists today — if the real data source is a different system entirely (e.g. a separate storefront/LMS whose data lives in Metabase), the mapping above needs the tech team to fill in the right column/field names on their side.

19. Screenshots

All captured live from the running application (freshly-seeded demo data, post-simplification), desktop viewport (1440×900). Saved under `docs/screenshots/`. Notice how short this list is now — that's the point of the simplification.

Login



Influencer — Sales Tracking, default state. Single nav item, sidebar-only "Sales Tracking."

Rahul Sharma
Influencer
Sign out

Influencer — Sales Tracking, with results (summary cards, course breakdown, UTM breakdown, detail table — all scoped to this influencer only)

Sales Tracking

Check the sales generated through your links and campaigns — no spreadsheets, no waiting on the team.

Start Date

06/01/2026

End Date

09/30/2026

UTM Source

All

UTM Medium

All

UTM Campaign

All

UTM Content

All

UTM Term

All

Coupon Code

All

Course Name

All

Apply Filters

Clear Filters

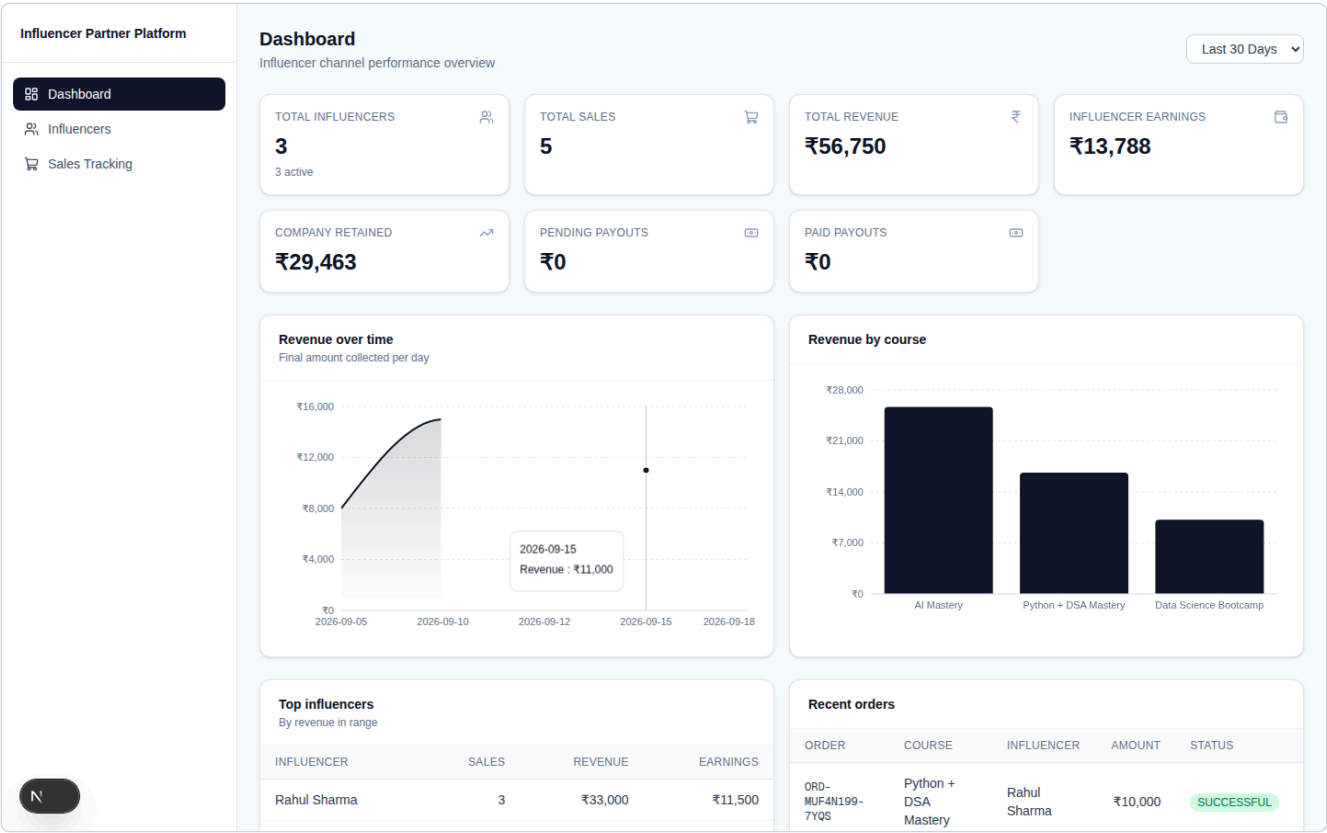
TOTAL SALES	TOTAL REVENUE	COURSES SOLD
35	₹4,15,700	3

Course-wise breakdown		
COURSE	SALES	REVENUE
AI Mastery	12	₹1,72,500
Data Science Bootcamp	11	₹1,27,200
Python + DSA Mastery	12	₹1,16,000

UTM content breakdown		
Which specific video/post/link is generating sales		
UTM CONTENT	SALES	REVENUE
video_03	9	₹1,13,150
sf	8	₹1,02,650
reel_01	8	₹94,700
video_02	5	₹55,700
video_01	5	₹49,500

Sales									
35 orders matching your filters									
DATE	COURSE	UTM SOURCE	UTM MEDIUM	UTM CAMPAIGN	UTM CONTENT	UTM TERM	COUPON	USER ID	ORDER ID
19 Sept 2026	Data Science Bootcamp	storefront	V_Upskill_Academy	Dedicated	video_01	PriyaVerma	PRIYA15	v_0960685053779	79578fa3514b978227393b2a
17 Sept 2026	Python + DSA Mastery	youtube	V_Upskill_Academy	FestiveSale	video_01	Aminakhan	—	v_6796889740224	5700b5cd42daf7e4ab6da72
17 Sept 2026	AI Mastery	instagram	influencer	FestiveSale	sf	HarshPriyam	—	v_1424600177635	8d6aeda2dd00651fc1087a1e
15 Sept 2026	Python + DSA Mastery	youtube	V_Upskill_Academy	FestiveSale	video_01	PriyaVerma	PRIYA15	v_6566811979637	b3ae62ed344756d9100aff59
14 Sept 2026	AI Mastery	youtube	influencer	Dedicated	video_03	Aminakhan	SAVE500	v_3578939717814	1904c87ec71861dab1461985
14 Sept 2026	Data Science Bootcamp	youtube	social	SeptemberLaunch	reel_01	PriyaVerma	RAHUL10	v_1698613339597	23b004e2d8ba9cd216252e36c
13 Sept 2026	Python + DSA Mastery	storefront	V_Upskill_Academy	SeptemberLaunch	video_02	HarshPriyam	RAHUL10	v_8023125063043	bc75bd634ee2aaf20a1491e4
06 Sept 2026	Python + DSA Mastery	instagram	social	SeptemberLaunch	reel_01	—	SAVE500	v_2022779741125	3010d5e3e387d6d54c8b041a
31 Aug 2026	AI Mastery	instagram	V_Upskill_Academy	Dedicated	sf	—	SAVE500	v_2718530907135	ff45714b858848175fb1a21d
31 Aug 2026	AI Mastery	storefront	influencer	SeptemberLaunch	sf	—	—	v_9163496844785	6b2c178f64e06d5a3e41f5df
28 Aug 2026	Python + DSA Mastery	storefront	V_Upskill_Academy	FestiveSale	video_01	PriyaVerma	—	v_93005407041217	e79a9aa2bfbd2357c12b9d05

Rahul Sharma Influencer [> Sign out]	18 Aug 2026	Python + DSA Mastery	storefront	V_Upskill_Academy	FestiveSale	video_01	—	SAVE500	v_1251480742920	5d924f69cc73b8f5f84c3236
	09 Aug 2026	Data Science Bootcamp	storefront	social	FestiveSale	reel_01	PriyaVerma	PRIYA15	v_4801202587700	c37822a8edf9ef9392941c6d
	04 Aug 2026	Python + DSA Mastery	youtube	social	FestiveSale	sf	—	SAVE500	v_6536968419864	6dde58eca76ccb95c5c0ac57
	03 Aug 2026	AI Mastery	storefront	V_Upskill_Academy	FestiveSale	video_02	—	—	v_9998519324768	59f87fff78073795a695660858
	31 Jul 2026	Python + DSA Mastery	storefront	V_Upskill_Academy	SeptemberLaunch	video_03	—	RAHUL10	v_4017893172814	45c4d77edf0bb80c2c3799f
	30 Jul 2026	AI Mastery	storefront	V_Upskill_Academy	Dedicated	sf	—	PRIYA15	v_3647857491281	ba80bce6ff3ebceec6715bc2
	30 Jul 2026	Data Science Bootcamp	youtube	social	FestiveSale	video_03	HarshPriyam	—	v_0192115240585	fc29c799618ccf57b11970cb
	29 Jul 2026	Data Science Bootcamp	storefront	social	FestiveSale	reel_01	—	—	v_0445764575827	59653998ccdec41f26e0503f
	28 Jul 2026	Data Science Bootcamp	instagram	V_Upskill_Academy	FestiveSale	video_02	HarshPriyam	PRIYA15	v_7850329532383	e1962f93fcd36820c9d7db0a
	25 Jul 2026	AI Mastery	instagram	social	SeptemberLaunch	video_03	—	—	v_3452238407745	c7ca0fcf1d1d7538c301b04c
	25 Jul 2026	AI Mastery	storefront	influencer	FestiveSale	video_03	HarshPriyam	RAHUL10	v_8091355512120	5ca8812db47788e990cd50b9
	22 Jul 2026	Data Science Bootcamp	storefront	social	SeptemberLaunch	video_03	—	SAVE500	v_5984657240631	a249c81563847b4252916d14
	16 Jul 2026	AI Mastery	instagram	V_Upskill_Academy	FestiveSale	reel_01	AminaKhan	PRIYA15	v_8674834655415	f114222b1a97fb026ba43056
	16 Jul 2026	AI Mastery	youtube	social	Dedicated	video_03	HarshPriyam	—	v_3142696542652	e6adada012b2eb9b4ed24843
	16 Jul 2026	AI Mastery	storefront	V_Upskill_Academy	Dedicated	sf	AminaKhan	PRIYA15	v_5539561367174	9e887f40a45162dd2159a9be
	12 Jul 2026	Data Science Bootcamp	instagram	V_Upskill_Academy	SeptemberLaunch	video_02	—	—	v_3169886783291	b15d6c9289e9f926d85b6d26
	09 Jul 2026	Data Science Bootcamp	youtube	V_Upskill_Academy	FestiveSale	reel_01	—	SAVE500	v_6144327001455	84aa866e6128405192ad22f8
	07 Jul 2026	AI Mastery	storefront	social	Dedicated	reel_01	PriyaVerma	—	v_7682928970374	fa98728301a3af24618dba35
	04 Jul 2026	Python + DSA Mastery	instagram	influencer	Dedicated	sf	—	RAHUL10	v_0038189617724	49cda72c7455d14105249ad0
	04 Jul 2026	Python + DSA Mastery	youtube	V_Upskill_Academy	FestiveSale	reel_01	—	—	v_3249544662764	beefc540ff91740bc8ad791e
	03 Jul 2026	Python + DSA Mastery	instagram	social	FestiveSale	video_02	AminaKhan	—	v_7893146166749	70d5e113c68ddd69bb02c8ba
	02 Jul 2026	Data Science Bootcamp	youtube	social	FestiveSale	sf	—	RAHUL10	v_6464097858750	18b70319401c0e1361137307
	28 Jun 2026	Data Science Bootcamp	youtube	influencer	Dedicated	video_03	AminaKhan	—	v_1990089798095	457820d03afb3825e20cf8d8
	27 Jun 2026	Python + DSA Mastery	youtube	V_Upskill_Academy	FestiveSale	video_03	—	PRIYA15	v_4793237926562	2a05ecaa66878f8b7a18ef21



Influencer Partner Platform

Dashboard

Influencers

Sales Tracking

Sales Tracking

Which influencer generated which sales, for which course, through which tracking source.

START DATE

01/01/2026

END DATE

12/31/2026

COURSE

All courses

CAMPAIGN

All campaigns

INFLUENCER

All influencers

COUPON CODE

RAHUL10

STATUS

All statuses

UTM SOURCE

youtube

UTM MEDIUM

influencer

UTM CAMPAIGN

dedicated

UTM CONTENT

sf

UTM TERM

harshpriyam

Apply filters

TOTAL SALES

5

TOTAL REVENUE

₹56,750

Sales by influencer

INFLUENCER	SALES	REVENUE
Rahul Sharma	3	₹33,000
Priya Verma	1	₹12,750
Amina Khan	1	₹11,000

Sales by course

COURSE	SALES	REVENUE
AI Mastery	2	₹27,750
Python + DSA Mastery	2	₹18,000
Data Science Bootcamp	1	₹11,000

Sales by UTM content

UTM CONTENT	SALES	REVENUE
rahul_video_1	1	₹15,000

Sales by coupon

COUPON	SALES	REVENUE
RAHUL10	2	₹18,000
PRIYA15	1	₹12,750
AMINA750	1	₹11,000

ORDER	DATE	COURSE	INFLUENCER	COUPON	UTM SOURCE / MEDIUM	CAMPAIGN / CONTENT / TERM	FINAL AMOUNT	EARNINGS	STATUS
ORD-MUF4N199-7YQS	18 Sept 2026, 12:00 am	Python + DSA Mastery	Rahul Sharma	RAHUL10	—	—	₹10,000	₹4,000	SUCCESSFUL
ORD-MUF4N18V-2LMW	15 Sept 2026, 12:00 am	Data Science Bootcamp	Amina Khan	AMINA750	—	—	₹11,000	₹750	PARTIALLY REFUNDED
ORD-MUF4N18F-51EZ	12 Sept 2026, 12:00 am	AI Mastery	Priya Verma	PRIYA15	—	—	₹12,750	₹1,913	SUCCESSFUL
ORD-MUF4N17T-ZN06	10 Sept 2026, 12:00 am	AI Mastery	Rahul Sharma	—	youtube / influencer	september_launch / rahul_video_1 / —	₹15,000	₹7,500	SUCCESSFUL
ORD-MUF4N16V-88I9	05 Sept 2026, 12:00 am	Python + DSA Mastery	Rahul Sharma	RAHUL10	—	—	₹8,000	₹3,200	REFUNDED

Admin

Admin

[> Sign out

No screenshots still need to be captured manually — all listed pages were reachable and captured directly from the live dev instance during this audit. Screenshots of the hidden-but-present pages (Courses, Coupons, Tracking Links, Agreements, etc.) were deliberately not included here since they're out of the current nav-visible scope — they're unchanged from before and still reachable by direct URL if needed.

20. User Flows

Login

Influencer/Admin → /login → Auth.js Credentials check → role-based redirect (/influencer or /admin)

Sales Tracking — Influencer

Influencer → Sales Tracking nav item (their only one) → select date range (+ optional filters) → Apply Filters
→ GET /api/influencer/sales-tracking → session-derived identity → mock adapter → results rendered

Sales Tracking — Admin

Admin → Sales Tracking nav item → optionally select an influencer + any filters → Apply filters
→ getSalesTrackingSummary() + listOrders() query the real Order/OrderAttribution tables
→ summary cards + 4 breakdown tables + paginated detail table rendered

```
Admin records a sale (manual entry, only path implemented today)
  → attribution resolved (coupon takes priority over tracking link)
  → applicable CommercialAgreementVersion resolved (coupon-linked > campaign-scoped > course-scoped > general)
  → calculateSplit() computes eligible revenue + influencer/company split (decimal.js)
  → FinancialTransaction (EARNING, PENDING) created with a full immutable snapshot of the terms used
  → Finance/Admin approves the transaction (PENDING → APPROVED)
  → Payout created, bundling approved transactions → marked PAID once money moves (external to the app)
```

Refund

```
Admin records a refund against an order
  → proportional reversal calculated against the original EARNING transaction
  → a REVERSAL transaction is created, linked back to the original via relatedTransactionId
```

21. Current Status

Feature	Status	Production Ready?	Tech Team Action Required?
Two-role model (Admin/Influencer)	Implemented — real migration, RBAC rewrite, seed data update	Yes	No — smoke-tested and rebuilt clean from scratch on a fresh DB
Auth / RBAC / row-level scoping	Implemented	Yes (pending secret rotation)	Rotate secrets; confirm session expiry policy
Influencer management	Implemented, nav-visible	Yes	No
Course/campaign/coupon/tracking-link management	Implemented, hidden from nav (not deleted)	Yes	No
Commercial agreements & financial calculation engine	Implemented, hidden from nav	Yes	No
Payouts	Implemented (record-keeping only), hidden from nav	Yes, if manual money movement is acceptable	Confirm whether a real payment-provider integration is needed
Sales Tracking — Influencer	Mock data	No	Implement real SalesTrackingAdapter (\$10)
Sales Tracking — Admin	Real DB data, nav-visible	Yes	No
Order ingestion	Manual entry only	Partially	Decide/build real ingestion path if manual entry isn't sufficient at scale
Marketing assets	Partially implemented	No file upload	Add upload capability if needed
CSV exports	Implemented	Yes at current scale (10k row cap)	Revisit if data volume grows significantly
Testing	One manual smoke script	No	Requires integration/CI test coverage before launch
Deployment/CI	Not implemented	No	Requires integration
Monitoring	Not implemented	No	Requires integration

22. Known Limitations

- Sales Tracking runs entirely on synthetic mock data — no real database table or Metabase connection exists yet.
- Order ingestion is manual-entry only; API/webhook/CSV-import paths are reserved in the schema (OrderSource enum) but not built.
- No payment gateway integration — payouts are recorded, not actually disbursed by the app.
- No email/WhatsApp/SMS notification delivery — notifications are in-app only.
- No file upload capability — marketing asset URLs must be supplied externally.
- No automated unit/integration test suite — one manual, DB-dependent Playwright smoke script only.
- No CI/CD pipeline.
- No monitoring/error-tracking integration (e.g. Sentry) — the only server-side logging is a single console.error in the shared API error handler.
- No password-reset flow, SSO, or MFA.
- next-auth is on a pre-1.0 beta release.
- legacy-portfolio/ at the repo root is unrelated leftover content from this repo's origin as a personal GitHub Pages site.

23. Testing

Existing tests: none in the traditional sense — no Jest/Vitest/Mocha, no *.test.ts / *.spec.ts files, no jest.config.* / playwright.config.*.

What does exist: scripts/e2e-smoke.mjs, run via npm run e2e:smoke. A plain Node script (not the Playwright test runner) that launches Chromium directly, logs in as the seeded influencer (rahul@creator.dev), and walks through RBAC enforcement, the full order → attribution → calculation → refund → approval → payout pipeline through the real UI, and the 4 CSV export endpoints. Requires a running dev server and a seeded database; not wired into any CI (none exists).

What's not covered: no coverage of the Sales Tracking feature specifically, no admin-side UI flows beyond what the smoke script touches, no unit-level coverage of the financial calculation engine's edge cases (e.g. HYBRID model thresholds, multi-partial-refund sequences).

Recommended pre-launch smoke-test checklist

```
[ ] Login (each role) / Logout
[ ] Role-based redirect and route gating (middleware)
[ ] Influencer cannot access another influencer's data via any UI action or direct API call
```

```
[ ] Sales Tracking: date filter, each UTM filter individually, coupon filter, course filter, combined filters
[ ] Sales Tracking: empty-result state, error state (simulate a backend failure)
[ ] Admin: create/edit an agreement, record a sale, approve earnings, create a payout, process a refund
[ ] CSV exports (all 4) produce correct, complete data
[ ] Mobile UI spot-check (drawer nav, filter grid reflow)
[ ] Security: attempt to pass another influencer's id/email as a parameter anywhere it's accepted client-side
```

24. Monitoring & Operations

Not currently implemented. No uptime monitoring, no APM, no error-tracking SDK, no structured logging — the only current error visibility is a single `console.error(err)` call inside the shared API error handler (`src/lib/api/handler.ts`), which will only reach whatever the hosting platform captures from `stdout/stderr`.

Recommended for the tech team to add before/at launch: application uptime monitoring, API error-rate tracking, database error/connection monitoring, auth failure tracking, and (once a real Sales Tracking adapter exists) connectivity/response-time monitoring for that data source.

25. Backup & Rollback

Not currently defined anywhere in the codebase — REQUIRES TECH TEAM CONFIRMATION for: - Database backup frequency/retention strategy. - Application deployment rollback approach (no CI/CD exists yet to define this against). - Database migration rollback plan (Prisma migrations are forward-only by default; a rollback plan needs to be decided by the team operating the production database).

26. Handover Summary

What is already built

A working platform now deliberately scoped to two roles (Admin, Influencer) and one core function (Sales Tracking) — Admin also manages influencers. The influencer-facing Sales Tracking (mock data, real UI/API) and the admin-facing Sales Tracking (real DB data, influencer/course/UTM-content/coupon breakdowns) are both nav-visible and both fully working. Underneath, the broader platform from an earlier build — auth + two-layer RBAC/data-isolation, commercial-agreement versioning, a decimal-precise financial calculation engine, attribution, tracking links, coupons, campaigns, courses, payouts, refunds/reversals, goals, notifications, audit logging, CSV exports — still exists, still works, and is hidden from navigation rather than deleted, so it can be brought back if scope widens again.

What the tech team needs to connect

A real production Postgres database; a real Sales Tracking data source (Metabase or otherwise) behind the existing `SalesTrackingAdapter` interface; a real order-ingestion path if manual entry isn't sufficient; production secrets and a hosting/CI/CD pipeline (none exist today).

What needs to be tested

Full RBAC/data-isolation re-verification against production data; the pre-launch smoke-test checklist (§23); a dedicated security pass (§14 Critical list).

What needs to be configured before launch

Environment variables and secrets (§8); hosting, domain, SSL (§15); database migrations-on-deploy; monitoring and backup strategy (§24–25).

Recommended for Phase 2

Real payment/LMS order ingestion (API or webhook); file upload for marketing assets; email/WhatsApp notification delivery; automated test suite + CI; password reset/SSO; pagination and export for the Sales Tracking detail table at scale.